

Practical Exploration of KNN Algorithm

Kristina Bhandari¹ and Namita Bhatta²

¹Department of Electronics and Computer Engineering, IOE, Thapathali Campus, Kathmandu 44600, Nepal

Corresponding author: Kristina Bhandari (e-mail: christinabhandari60@gmail.com).

ABSTRACT K-Nearest Neighbor (KNN) algorithm is a supervised machine learning algorithm predominantly used for both classification and regression purpose. It classifies datasets by comparing the testing query to the k nearest training data points and performs majority voting to determine the final classification. KNN is particularly effective in pattern recognition due to its non-parametric nature, which allows it to adapt to data without assuming an underlying distribution. In this paper, we explore the application of K-Nearest Neighbors in the context of recognizing handwritten digits using the Pen-Based Recognition of Handwritten Digits dataset. This dataset comprises various attributes related to the coordinates of pen strokes captured during the digit writing process. The goal is to investigate whether the provided attributes can reliably predict the digit represented by the pen strokes. This research seeks to leverage the KNN algorithm's ability to classify instances based on their proximity to existing data points. By measuring the distance between instances in the dataset and their k-nearest neighbors using Euclidean distance, Manhattan distance and Chebyshev distance metrics, we aim to develop a predictive model that effectively classifies handwritten digits into their correct categories.

INDEX TERMS Chebyhev Distance, Classification, Euclidean Distance, Handwritten digits, K-Nearest Neighbour, Manhattan Distance

I. INTRODUCTION

The K-Nearest Neighbors (KNN) algorithm is widely recognized in machine learning for its simplicity and effectiveness in tackling classification and regression tasks. Unlike many other algorithms that create explicit rules or boundaries between classes, KNN takes a different approach. It works by comparing new data points with previously collected data points from a dataset. By identifying the most similar data points, known as nearest neighbors, KNN predicts the class of the new data point based on the majority class among its neighbors. One thing that sets KNN apart is its simple methodology. It works on the tenet that comparable items tend to be grouped together rather than establishing rigid rules for classification. This makes it especially appropriate for applications where precise boundaries are not as critical as the

relationships between data elements. In recommendation systems, for instance, KNN can suggest films or goods based on user tastes that are similar to its own. Similar to this, KNN can help anticipate a patient's condition in medical diagnosis by comparing their symptoms to those of patients who have established diagnoses. It was introduced by Evelyn Fix and Joseph Hodges in the early 1950s for nearest neighbor classification. It was formalized as "K-nearest neighbors" by Thomas Cover and Peter Hart in 1967, becoming a cornerstone in machine learning for its simplicity and versatility in tasks like pattern recognition and recommendation systems.

In practical terms, KNN's simplicity makes it easy to understand and implement. It does not require

complex parameter tuning or extensive training time, making it accessible even to those without deep technical expertise. Despite its simplicity, KNN remains a powerful tool in the machine learning toolkit, valued for its ability to handle diverse datasets and provide interpretable results. Whether used for pattern recognition, anomaly detection, or data mining, KNN continues to demonstrate its versatility and relevance across various domains where understanding the relationships between data points is key.

II. RELATED WORK

Pádraig Cunningham et al. [1] provides a comprehensive overview of key aspects such as distance metrics, computational complexity, and dimension reduction. It discusses various similarity metrics, including Cosine Similarity, Correlation, Earth Mover's Distance (EMD), and Dynamic Time Warping (DTW), highlighting their relevance to different data types. The report addresses the computational challenges of K-NN with large datasets, proposing solutions like Kd-Trees, Ball Trees, and Approximate K-NN methods such as Locality Sensitive Hashing (LSH) and Annoy to enhance efficiency. Additionally, it covers dimension reduction techniques like Principal Component Analysis (PCA) and supervised feature selection to improve performance in high-dimensional spaces. The conclusion emphasizes the importance of selecting appropriate distance metrics and dimension reduction methods to balance efficiency and accuracy in K-NN applications.

Jingwen Sun et al. [2] highlights its significance as a simple yet effective non-parametric method widely used in pattern recognition, text classification, and various other fields. The paper begins by detailing the basic principles, ideas, implementation steps, and Java code for KNN, followed by a discussion on its shortcomings and potential improvements. It also explores how KNN handles multi-label data and its application in text classification, emphasizing its performance in this domain. The algorithm's core idea involves calculating the distance between data points and classifying a point based on the majority class among its k-nearest neighbors. Despite its

simplicity, KNN faces challenges like optimal k-value selection, distance metric choice, and handling imbalanced data distributions, which the paper addresses through various improvement schemes.

Shahadat Uddin et al. [3] examines various K-Nearest Neighbor (KNN) algorithm variants to determine their effectiveness in predicting disease risk. The analysis identified the Hassanat, ensemble approach, and generalized mean distance as the most suitable KNN variants due to their high accuracy, precision, and recall measures. The ensemble approach KNN variant, in particular, achieved the highest performance across multiple metrics, making it the best choice for handling medical datasets. The study highlights how these variants address the limitations of the classic KNN algorithm and prove their potential in the medical domain, demonstrating adaptability and potential for further enhancement.

Gongde Guo et al. [4] addresses the limitations of the k-Nearest Neighbours (KNN) classification method, which include low efficiency and dependence on selecting an appropriate value for k. The authors propose a novel KNN-based method that constructs a model from the data to improve classification efficiency and accuracy. This method automatically determines the optimal k value for different data sets and reduces dependency on k. Experimental results on public datasets show that this model-based approach performs comparably to C5.0 and standard KNN in terms of accuracy but is more efficient. The paper discusses related research, presents the modeling and classification algorithm, and evaluates the method through experiments.

III. METHODOLOGY

A. THEORY

The K-Nearest Neighbors (KNN) algorithm operates by categorizing data points based on their proximity to other points within the dataset. By specifying a number of neighbors (K), KNN assigns a class to each data point by determining the majority class among its closest neighbors. Each data point in KNN is depicted as a coordinate in a multi-dimensional space, and its

classification relies on the classes of nearby points. This method is particularly advantageous when dealing with tasks where class boundaries are uncertain or irregular. KNN's simplicity makes it easy to interpret: the classification of a data point is influenced by the classes of its nearest neighbors. This straightforward approach is beneficial in fields such as medical diagnosis or financial risk assessment, where transparency and clarity are crucial. By selecting relevant features and specifying the number of neighbors (K), KNN effectively groups data points into classes based on their proximity in the feature space. Its ability to adapt to different datasets and its straightforward interpretation of results highlight KNN's usefulness in scenarios requiring a clear understanding of the decision-making process.

B. MATHEMATICAL FORMULAE

1. Calculation of Euclidean Distance

Euclidean distance is the straight-line distance between two points in a Euclidean space. For two points $p = (p_1, p_2, \dots, p_n)$ and $q = (q_1, q_2, \dots, q_n)$ the Euclidean distance d is calculated as:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

2. Calculation of Manhattan Distance

Manhattan distance (also known as city block distance or taxicab distance) is the sum of the absolute differences between the coordinates of two points. For two points $p = (p_1, p_2, \dots, p_n)$ and $q = (q_1, q_2, \dots, q_n)$ the Manhattan distance d is calculated as:

$$d(p, q) = \sum_{i=1}^n |p_i - q_i|$$

3. Calculation of Chebyshev distance

Chebyshev distance (or maximum metric) is the maximum absolute difference between the coordinates of two points. For two points $p = (p_1, p_2, \dots, p_n)$ and $q = (q_1, q_2, \dots, q_n)$ the Chebyshev distance d is calculated as:

$$d(p, q) = \max(|q_i - p_i|)$$

4. KNN Selection Criteria

Let F be a distance function. $x_1, x_2, x_3, \dots, x_n$ are training examples. The class $C(v)$ for a training example v is determined as:

$$C(v) = \arg \min_{x \in \{x_1, \dots, x_n\}} (F(v, x))$$

5. Performance Evaluation

For performance evaluation of the model, we have used classification report that summarizes metrics that assess how well the model has performed in classifying different classes or categories. Here are some metrics used:

Precision: Precision is a metric used in statistics and machine learning to evaluate the performance of a classification model, especially for binary classification tasks.

$$Precision = \frac{TP}{TP + FP}$$

Where,

- True Positives (TP) are the number of correctly predicted positive instances.
- False Positives (FP) are the number of incorrectly predicted positive instances (instances predicted as positive but are actually negative).

Accuracy: In machine learning and statistics, accuracy is a measure of a model's overall performance in producing accurate predictions. It calculates the percentage of all predictions produced by the model that come true. The formula for calculating accuracy is:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where,

- TP (True Positives): These are the instances that are actually positive and were correctly predicted as positive by the model.
- TN (True Negatives): These are the instances that are actually negative and were correctly predicted as negative by the model.

- FP (False Positives): These are the instances that are actually negative but were incorrectly predicted as positive by the model.
- FN (False Negatives): These are the instances that are actually positive but were incorrectly predicted as negative by the model.

F1 Score: The F1 score is a metric used to evaluate the performance of a classification model, particularly when dealing with imbalanced classes. It combines precision and recall into a single metric and provides a balance between these two measures.

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Where,

- Precision quantifies the model's accuracy in making positive predictions. This can be expressed as the ratio of true positives, or positively anticipated cases, to the total of false positives, or positively predicted instances that were mistakenly forecasted.
- Recall, which is often referred to as sensitivity or true positive rate, quantifies the model's accuracy in identifying positive cases. It is the proportion of real positives to the total of both true positives and false negatives, or cases that were misclassified as negative.

Macro-average: It calculates the metric independently for each class and then takes the average of those values. It is calculated as:

$$Macro - average = \frac{1}{N} \sum_{i=1}^N Metric_i$$

Weighted average: It calculates the metric for each class and then takes a weighted average based on the number of instances of each class. It gives more weight to classes with more instances, making it useful in imbalanced datasets.

Weighted average

$$= \frac{1}{Total\ instances} \sum_{i=1}^N (Metric_i \times Support_i)$$

Support: It refers to the number of occurrences of each class in the actual dataset. It helps in understanding how balanced or imbalanced the dataset is across different classes.

$Support_i = \text{Number of instances of class } i$

C. SYSTEM BLOCK DIAGRAM

Figure 1 illustrates the process of k-Nearest Neighbours algorithm for classification tasks. It begins with the k-Nearest Neighbors (KNN) algorithm. It begins with the dataset, which undergoes mean adjustment and normalization by standard deviation to standardize the features. After that, the dataset is split into training and testing sets. Then, the KNN model is initialized, and the training dataset is fitted to the model. The model's performance is then evaluated. If the desired accuracy is not achieved, the algorithm iterates through a loop where it starts by selecting one neighbor, evaluates performance, and then increases the number of neighbors incrementally. This process continues, testing the model with the new number of neighbors each time, until the desired accuracy is reached. Once the optimal number of neighbors is determined, the KNN model is considered trained. Testing data points are then used to obtain the KNN, and the algorithm selects the predominant class among the neighbors as the target class for classification. The performance evaluation ensures that the model's accuracy is satisfactory before finalizing the trained model.

D. INSTRUMENTATION TOOLS

The entire process was carried out using Python within the Google Colaboratory platform, which is a convenient online tool for coding and running data analysis tasks. We used several Python library to tackle the problem. We began by loading our dataset and using Pandas to visualize and understand its structure. The NumPy library was employed for efficient numerical operations and array manipulation, allowing for seamless handling of

numerical data. Constructing the k-Nearest Neighbors (k-NN) classifier involved employing various functions from the scikit-learn library. This foundational machine learning algorithm learns from data to predict outcomes based on similarities to other data points. Visualizing the results of our k-NN model was crucial for assessing its performance. To achieve this, we utilized visualization tools such as Seaborn and matplotlib, creating clear and informative graphs and charts. Finally, we evaluated the effectiveness of our k-NN classifier using key metrics such as the confusion matrix, accuracy, precision, recall, and F1 score. These metrics provided a comprehensive assessment of how well our model predicted outcomes based on the dataset.

E. WORKING PRINCIPLE

1. Dataset preparation

The 'Pen-Based Recognition of Handwritten Digits' dataset was taken from the UC Irvine Machine Learning Repository. It consists of 10,992 instances with 16 integer features along with target class, each representing coordinates and other measurements captured from handwritten digits. Firstly, the dataset is loaded from the scikit-learn library. The feature names, the target classes and the attributes are then loaded to a Pandas dataframe and the values are visualized. We first check if there is class imbalance, in this case there is not as seen in **Figure 2**. We performed feature scaling using standardization to ensure that all features contribute equally to distance calculations, preventing features with larger ranges from dominating. This process scales the features to have a mean of 0 and a standard deviation of 1. Also, we used correlation to identify how features are positively correlated with class variable. We also tried techniques other than correlation for feature selection, like Information Gain to possibly enhancing model accuracy and efficiency. Finally, the dataset was split into training and validation sets in an 80:20 ratio to evaluate the model's performance effectively.

2. KNN Model Training

For training a k-Nearest Neighbours (k-NN) classifier, the first process involves splitting the dataset into training and validation sets using scikit-learn. The classifier is then instantiated with a specified number

of neighbors (K). During training, the algorithm stores the entire training dataset, enabling it to find the K nearest neighbors for each new data point in the validation set. This is achieved using a chosen distance metric such as Euclidean, Manhattan, or Chebyshev distance. Classification predictions are made by determining the majority class among these neighbors. Subsequently, the classifier applies these predictions to the validation set to predict labels, which are then compared against the true labels to evaluate its performance.

In this study, k-fold cross-validation was used to determine the optimal number of neighbors (k) for the KNN classifier. By evaluating the model's accuracy across different values of k, we identified the k value that achieved the best performance, ensuring the model is both accurate and generalizable. We used k-fold cross-validation, specifically with 10 folds, to systematically evaluate the performance of the KNN classifier across different values of k and the accuracy scores for k values ranging from 1 to 29 in steps of 2 were calculated.

3. Data Analysis and Visualization

After training the k-Nearest Neighbors (k-NN) classifier, the dataset is visualized using various plots. Accuracy is plotted against different k value to visualize the results of k-fold cross validation, and bar plots are obtained. Confusion matrices are used to visualize the performance of the model and its outcomes.

IV. RESULTS

A. DEFAULT KNN CLASSIFIER

The default KNN classifier demonstrated outstanding performance on the digit recognition dataset, which was split into an 80-20 train-test ratio with a random state of 43 to ensure reproducibility. Using the default parameters (`n_neighbors=5`, and `metric='minkowski'`), the classifier achieved an impressive accuracy of 99%. The KNN classifier achieved high accuracies across different distance metrics: 99.27% with Euclidean, 99.32% with Manhattan, and 98.73% with Chebyshev. The high precision, recall, and F1-scores across all classes, each close to 1.00, highlight the model's effectiveness in accurately identifying each digit

with minimal misclassifications. The confusion matrix further illustrates the classifier's robustness, showing most predictions on the diagonal, indicating correct classifications, with very few off-diagonal entries as shown in **Figure 3**.

B. KNN CLASSIFIER WITH RANDOM WEIGHTS

The KNN classifier with randomly initialized weights using the numpy's random.random function also demonstrates high performance on the digit recognition dataset. The dataset was split into an 80-20 train-test ratio with a random state of 43, and the randomly weighted KNN model achieved an overall accuracy of 99%, similar to the default KNN model. The classification report indicates high precision, recall, and F1-scores for all classes, each close to 1.00, suggesting the model's robustness in correctly identifying the digit classes with minimal errors as shown in **Figure 4**.

C. KNN CLASSIFIER WITH RANDOM WEIGHTS ON TOP 4 FEATURES

The KNN classifier with the top 4 features (features 0, 2, 1, and 3) selected by Pearson correlation and randomly assigned weights performs significantly worse than the default KNN classifier with all features. The substantial drop in accuracy to 64% and the wide variability in precision, recall, and F1-scores across classes indicate that reducing the feature set and using random weights negatively impacted the model's ability to accurately classify the digits. This suggests that using all available features and uniform weights is more effective for this particular digit recognition task, highlighting the importance of comprehensive feature selection and appropriate weight assignment in achieving optimal model performance.

D. OPTIMUM KNN CLASSIFIER WITH TOP 4 FEATURES

Using the top 4 features identified by Pearson correlation (features 8, 2, 1, and 3), as shown in **Figure 5**, the weighted KNN model was optimized using a range of k values, and the optimal k was found to be 17, yielding a best accuracy of 66.02%

that can be seen in **Table 1**. Despite this optimization, the performance remained relatively modest. The confusion matrix illustrates a notable number of misclassifications across various classes. For instance, Class 0 had a recall of 64% with 28 instances misclassified as Class 3, and Class 2 had a recall of 56% with significant misclassifications into Class 3 and Class 5. The overall precision, recall, and F1-scores were fairly balanced but reflected the model's struggle to achieve high accuracy, with values around 0.66.

Using Information Gain for feature selection, the top 4 features identified were features 5, 7, 15, and 13 as shown in **Figure 5**. The KNN model was optimized using a range of values, with the optimal k found to be 17, achieving a best accuracy of 76.32%.

Using ANOVA-F test for feature selection, the top 4 features were features 1, 4, 13 and 15 as shown in **Figure 10**. The optimal k found to be 9, achieving a best accuracy of 82.54%.

V. DISCUSSION AND ANALYSIS

For 5 nearest neighbors, the KNN classifier achieved high accuracies across different distance metrics: 99.27% with Euclidean, 99.32% with Manhattan, and 98.73% with Chebyshev. The similarities in performance suggest that this dataset's features are well-discriminated, allowing various distance metrics to effectively distinguish between classes. The minor differences in accuracy indicate that while the dataset is robust to changes in the distance metric, Manhattan distance slightly outperforms others in capturing the nuances of the data.

An important takeaway from the KNN classifier results is the impact of the number of neighbors (k) on model performance. Our evaluation showed that increasing the k value generally led to better accuracy in distinguishing digit classes. However, it is essential to find the right balance, as too high a k value can introduce noise or irrelevant patterns from the data. Thus, selecting an optimal k is crucial for achieving a high-performing model. This is illustrated by our findings where the optimal k value of 17 for the weighted KNN with top 4 features

selected using Information Gain resulted in an accuracy of 76.33%. The confusion matrix is shown in **Figure 8** and the accuracy is shown in **Table 2**. And, the optimal k value of 9 for the weighted KNN with top 4 features selected using ANOVA-F test resulted in an accuracy of 82.59%. The confusion matrix is shown in **Figure 11** and the accuracy is shown in **Table 3**. However, when we used correlation for feature selection, we got an accuracy of just 64% on the basis of Euclidean distance and 65% in the basis of Manhattan distance. The difference in accuracy between the two feature selection methods stems from how they evaluate feature importance. Pearson correlation measures linear relationships, which may miss non-linear interactions vital for classification, leading to less informative feature selection. In contrast, Information Gain assesses how much each feature reduces uncertainty about the target variable, capturing more relevant aspects for classification. Hence, feature selection by information gain leads to the selection of features that more effectively contribute to distinguishing between classes, hence the higher accuracy achieved with Information Gain.

Another thing to note is that feature scaling is a crucial step in the preprocessing pipeline, especially for algorithms like K-Nearest Neighbors (KNN) that are based on distance metrics. In our analysis, feature scaling was necessary to ensure that all features contributed equally to the distance calculations, thereby preventing any single feature from disproportionately influencing the model's performance. The Pearson correlation plot in **Figure 5** shows the correlation coefficients between each feature and the target digit class. Features 0, 2, and 3 have the highest positive correlations with the digit class, indicating that these features have a stronger linear relationship with the target variable and are more informative for the classification task.

Hence, the KNN classifier allows for an in-depth analysis of feature importance. By examining the attributes of the nearest neighbors that influence classification, we can pinpoint key features that affect model performance. For example, selecting the top features using Information Gain (features 5,

7, 15, and 13) significantly improved model accuracy compared to using features selected by Pearson correlation. Understanding feature importance helps in comprehending the underlying factors that drive the classification task.

Figure 9 shows the performance of the K-Nearest Neighbors (KNN) classifier using different values for the number of neighbors (k). Initially, as k increases, the model becomes less sensitive to noise, leading to improved accuracy. This is because considering more neighbors helps in smoothing out the influence of any single noisy or outlier point. However, after a certain point, increasing k further does not significantly improve accuracy because the model starts to average over more points, which might include less relevant or more distant neighbors, leading to a slight decrease or stabilization in accuracy. With very low k values (e.g., $k=1$), the model has high variance and low bias, so it can overfit the training data and be very sensitive to individual data points. As k increases, the model's variance decreases, and bias increases, leading to a more balanced and generalizable model. For this dataset, a k value of 17 provides the best accuracy. Beyond this point, additional neighbors do not contribute to significant improvements.

VI. CONCLUSION

In this study, we applied the K-Nearest Neighbors (KNN) classifier to the Pen-Based Recognition of Handwritten Digits dataset, aiming to evaluate its effectiveness and explore the impact of various factors on its performance.

The default KNN model, using all features and no weighting, achieved an impressive accuracy of 99%. This highlights the classifier's ability to effectively utilize the entire feature set to distinguish between different digit classes. However, when random weights were applied to the features, the accuracy remained similarly high, indicating the robustness of the dataset and the model's capability to handle feature weighting without significant performance degradation.

Feature selection plays a critical role in the performance of the KNN classifier. Hence, the KNN

Classifier allows for the assessment of feature importance, shedding light on the pivotal attributes that influence the classification process, specifically tailored to digit classification.

The cross-validation process was crucial in identifying the optimal k value for the KNN classifier. By systematically evaluating different k values, we ensured that the chosen k of 9 provides the best performance for this specific dataset. This process highlights the importance of cross-validation in model tuning, ensuring that the model is both accurate and generalizable to new data. It underscores the delicate balance required between bias and variance to achieve optimal classification results.

Choosing the optimal number of neighbors (k) in KNN is crucial. A too-small k makes the model

overly sensitive to noise, causing overfitting, while a too-large k overly generalizes predictions, causing underfitting. Thus, balancing k ensures the model is accurate and generalizable.

While the KNN classifier proved effective in this context, it is not without limitations. The curse of dimensionality can impact its performance in higher-dimensional spaces, making feature selection and dimensionality reduction crucial steps. Additionally, the computational cost of the KNN algorithm increases with larger datasets and higher dimensions, necessitating efficient implementation and potential use of approximation techniques.

Future research could explore advanced feature selection methods, such as Recursive Feature Elimination (RFE) or Principal Component Analysis (PCA), to further enhance the model's performance

REFERENCE

- [1] P. Cunningham and S. J. Delany, "k-Nearest Neighbour Classifiers," 2021.
- [2] J. Sun, W. Du and N. Shi, "A Survey of kNN Algorithm," 2018.
- [3] S. Uddin, I. Haque, H. Lu, M. A. Moni and E. Gide, "Comparative performance analysis of K-nearest neighbour (KNN) algorithm and its different variants for disease prediction," 2022.
- [4] G. Guo, H. Wang, D. A. Bell and Y. Bi, "KNN Model-Based Approach in Classification," 2004.

BIOGRAPHY



KRISTINA BHANDARI, a fourth year computer engineering undergraduate student at IOE, Thapathali Campus. She is eager to explore the latest advancements in AI and aims to make significant contributions to the field. (THA077BCT022)



NAMITA BHATTA, a fourth-year computer engineering undergraduate at IOE, Thapathali Campus. Her goal is to leverage her knowledge and expertise in AI to make a significant impact in the field. (THA077BCT048)

APPENDIX

I. FIGURES

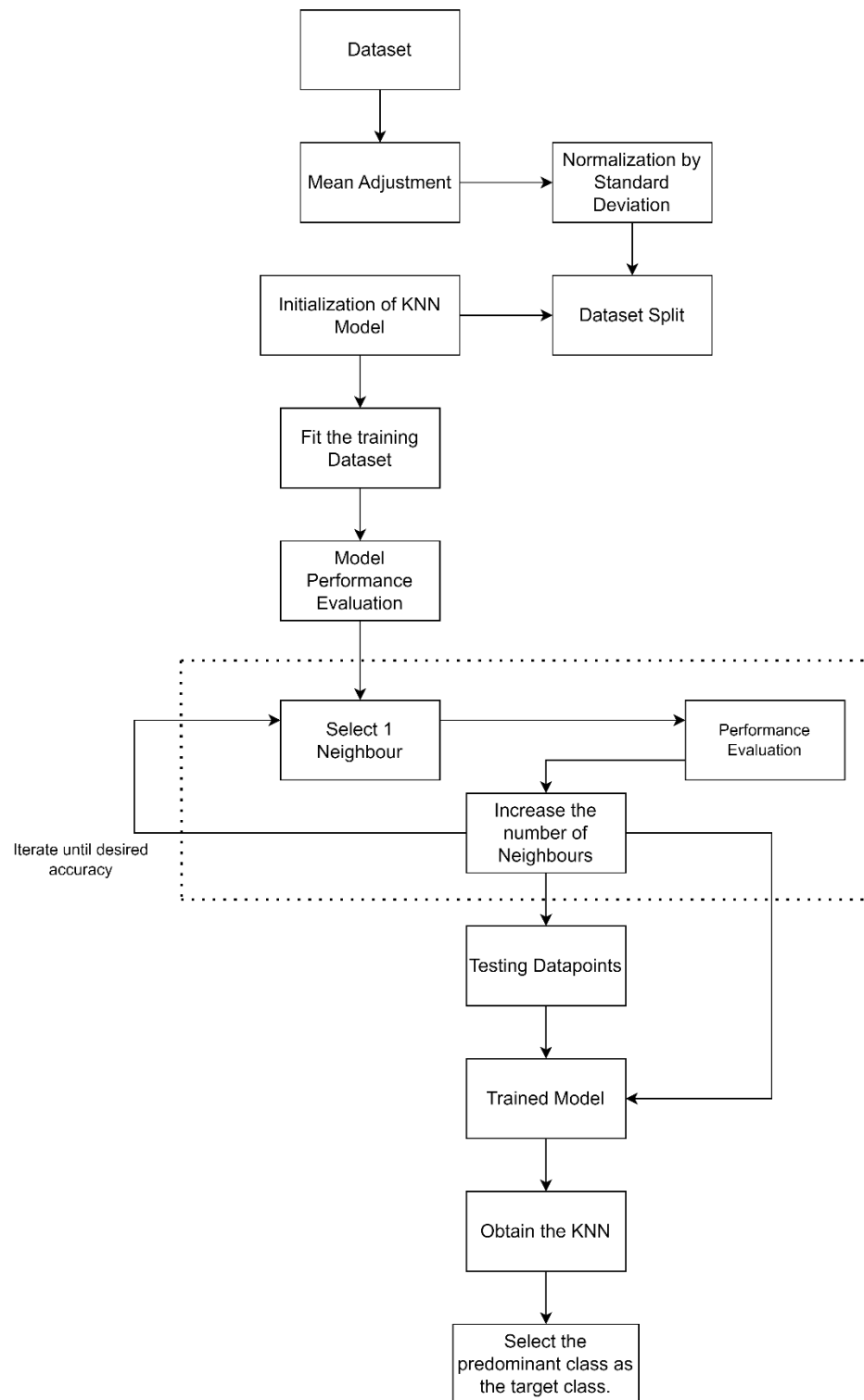


Figure 1 System Block diagram of KNN Classifier

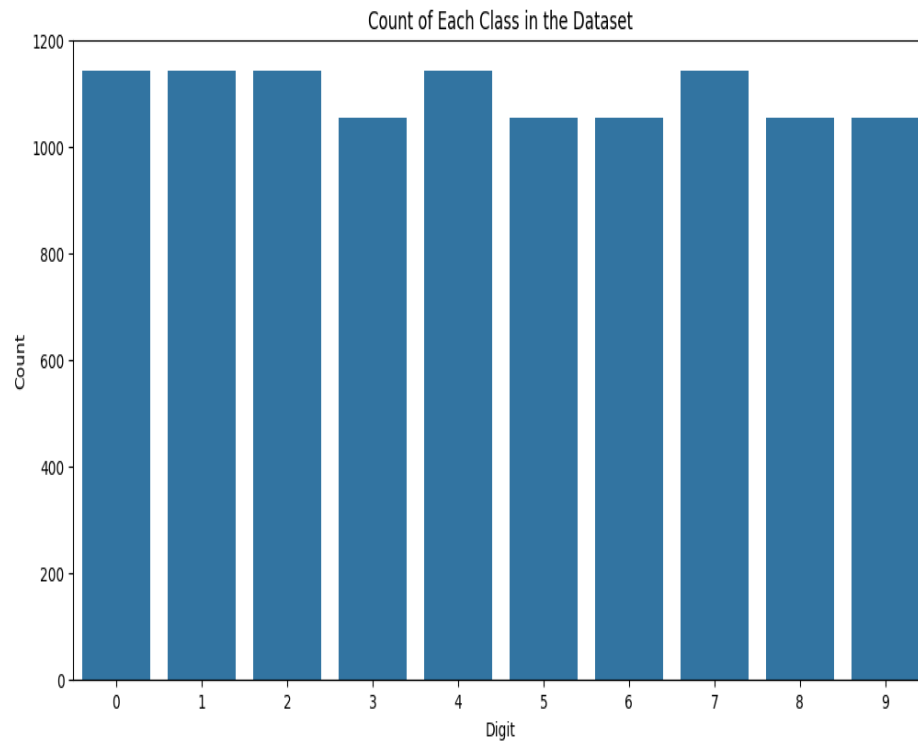


Figure 2 Class distribution

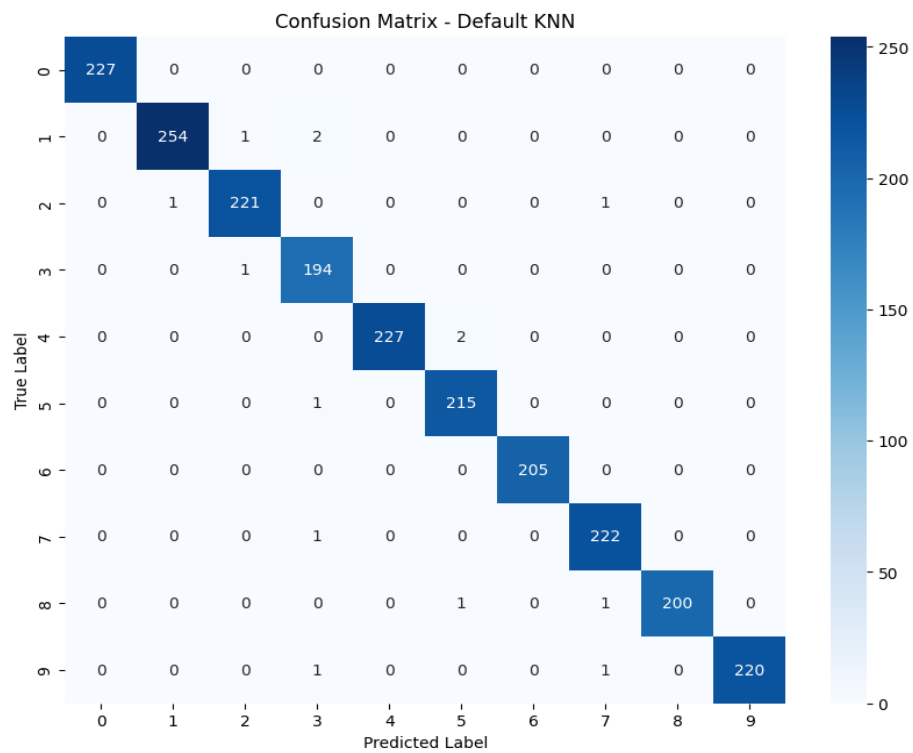


Figure 3 Confusion matrix of default kNN model

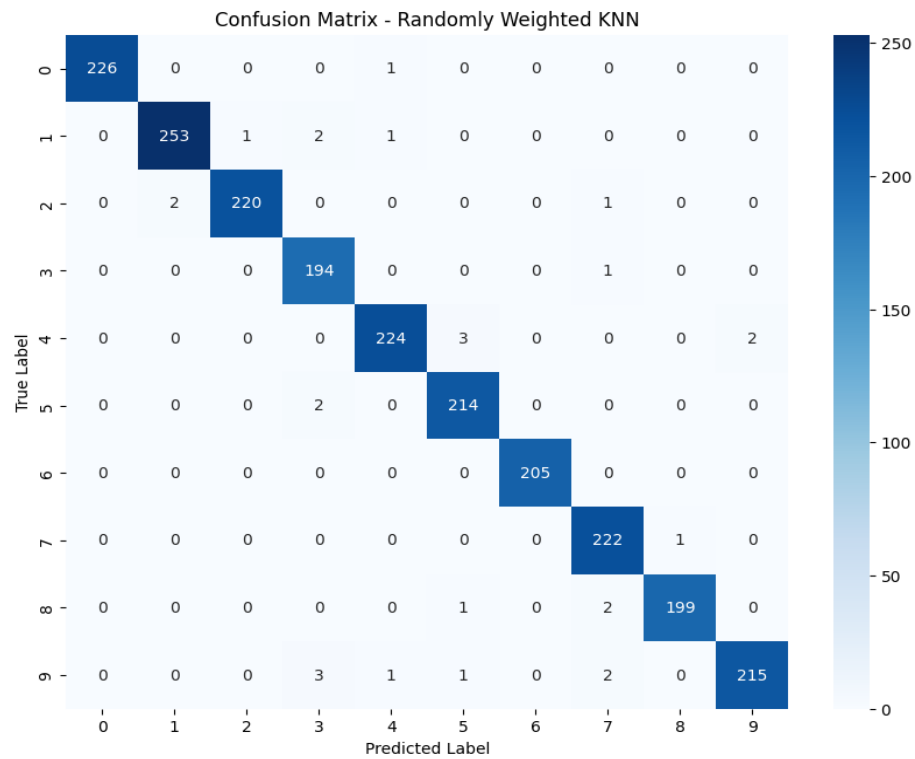


Figure 4 Confusion matrix of kNN classifier with random weights

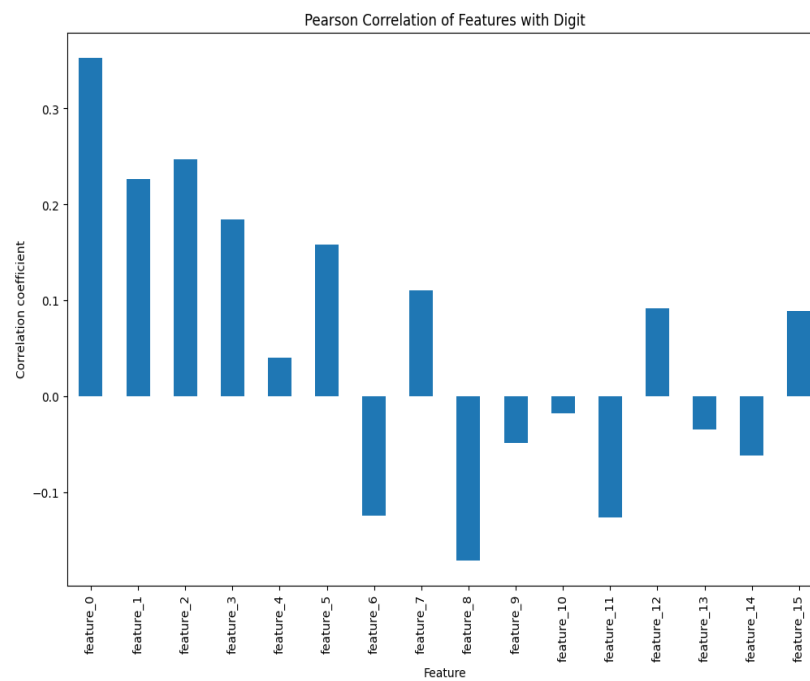


Figure 5 Correlation of features with target class

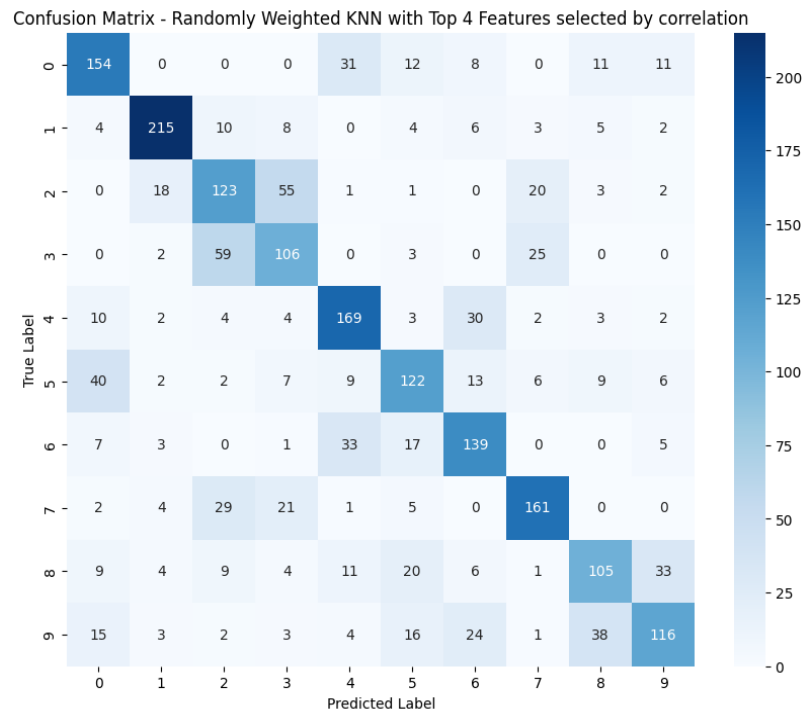


Figure 6 Weighted kNN with optimal $k=15$ using top 4 features selected by correlation analysis

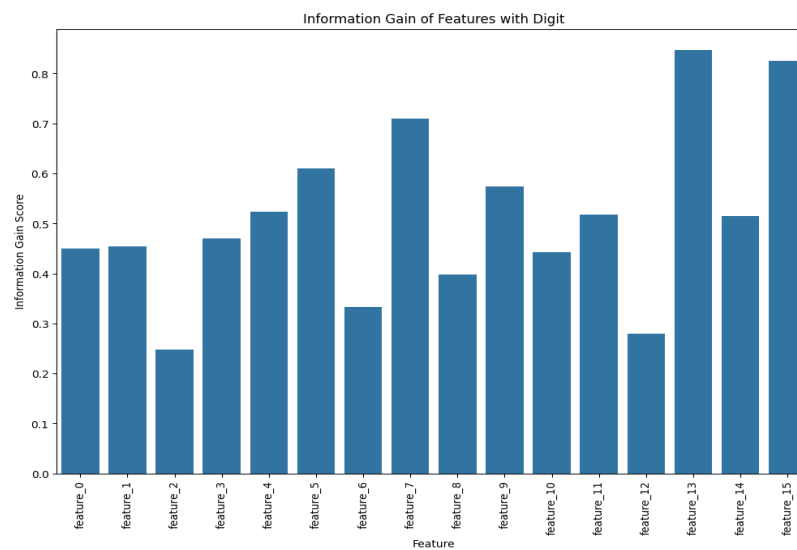


Figure 7 Information gain of features with target class

Confusion Matrix - Weighted KNN with Optimal k and Top 4 Features Selected by Information Gain

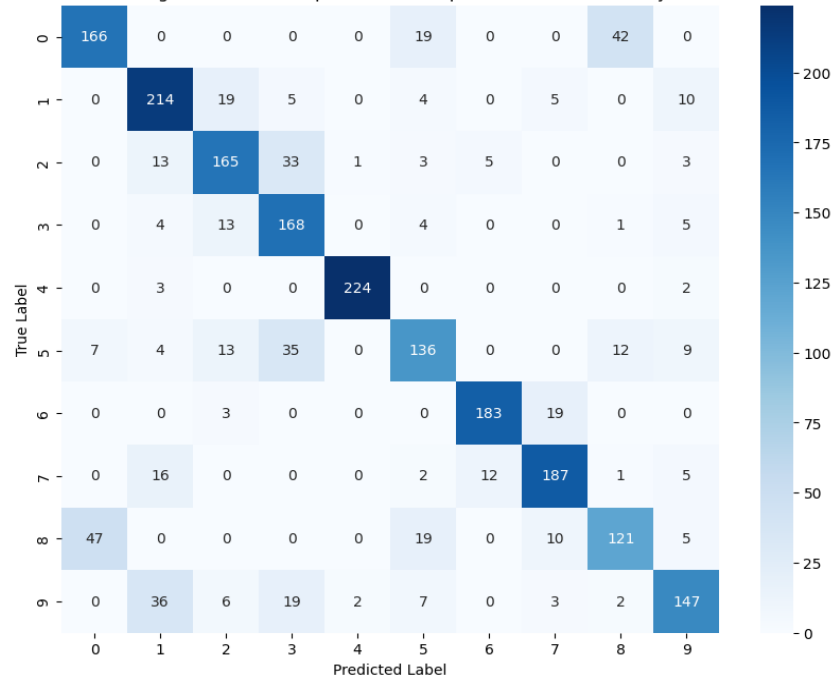


Figure 8 Weighted kNN with optimal k=17 using top 4 features selected by IG

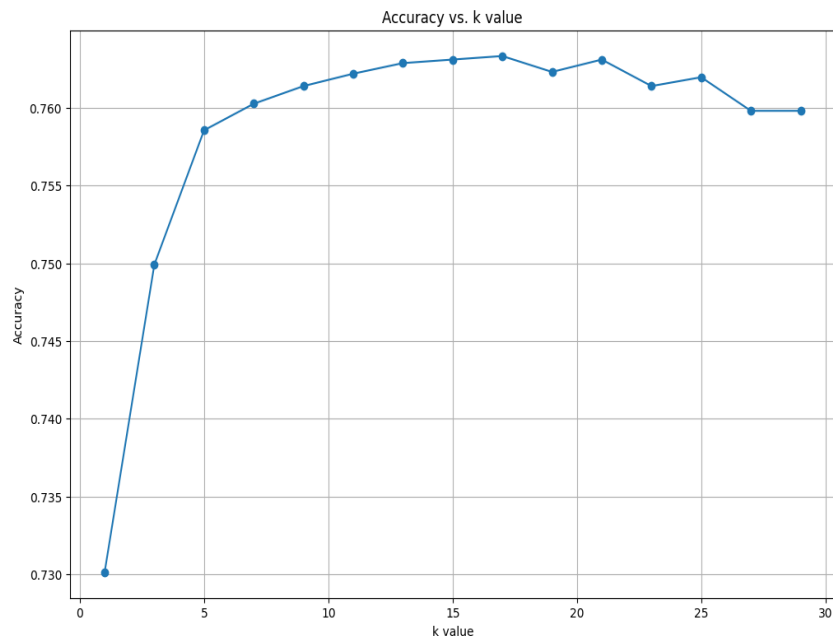


Figure 9 Accuracy vs k-value for optimum k value

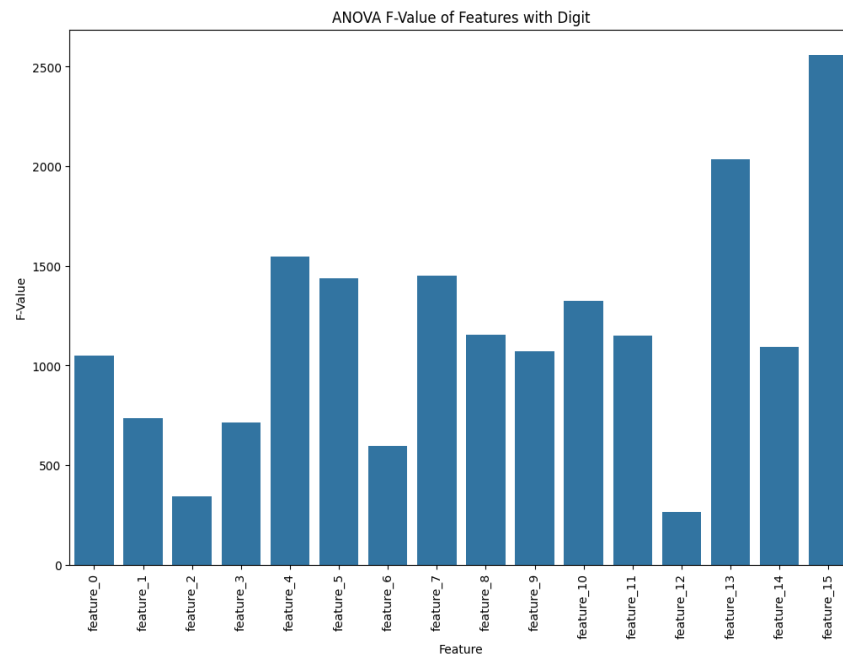


Figure 10 ANOVA-F values of features with target class using IG for feature selection

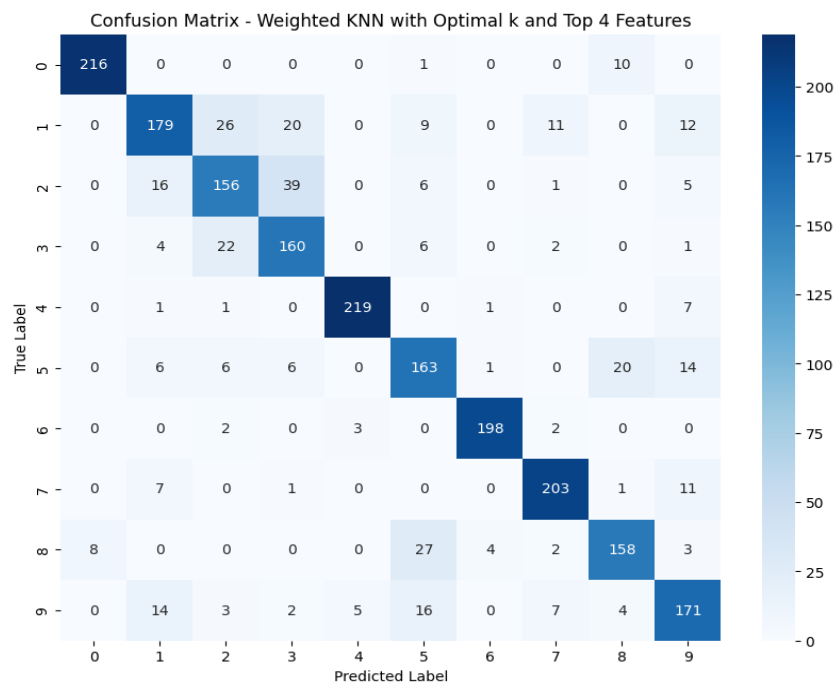


Figure 11 Confusion matrix for optimal $k=9$ using top 4 features selected by ANOVA-F test

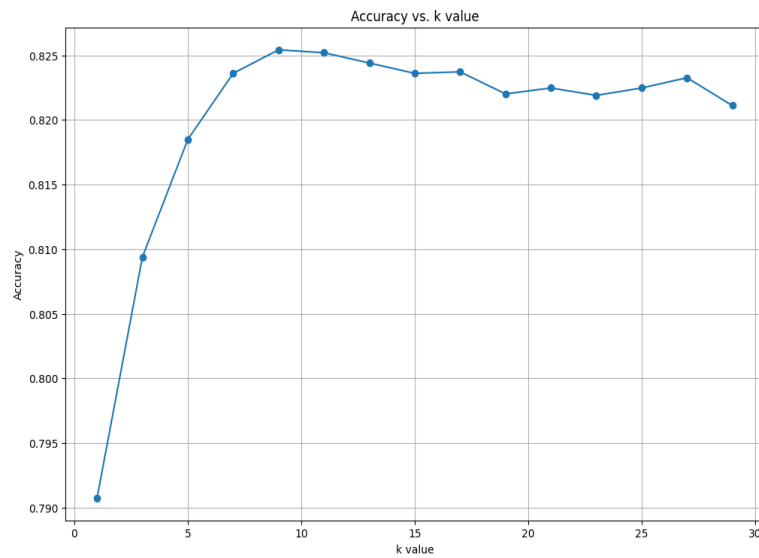


Figure 12 Accuracy vs k-value for optimum k value using ANOVA-F test for feature selection

II. TABLES

Table 1. Accuracy at different values of k using correlation analysis for feature selection

k	Average Accuracy
1	0.5848
3	0.6179
5	0.6371
7	0.6468
9	0.6506
11	0.6557
13	0.6578
15	0.66.02
17	0.6586
19	0.6590
21	0.6594
23	0.6596
25	0.6579
27	0.6570
29	0.6573

Table 2 Accuracy at different values of k using Information Gain for feature selection

k	Average Accuracy
1	0.7301
3	0.7499
5	0.7586
7	0.7603
9	0.7614
11	0.7622
13	0.7629
15	0.7631
17	0.7633
19	0.7623
21	0.7631
23	0.7614
25	0.7620
27	0.7598

Table 3 Accuracy at different values of k using ANOVA-F test for feature selection

k	Average Accuracy
1	0.7907
3	0.8094
5	0.8185
7	0.8236
9	0.8254
11	0.8252
13	0.8244
15	0.8236
17	0.8237
19	0.8220
21	0.8225
23	0.8219
25	0.8225
27	0.8233
29	0.8211